

Aula 05 - 17/03/2026

Objetos, instâncias e construtores

Atributos de instância

Existem no **contexto de uma instância** e só podem ser acessados através de um **objeto instanciado**:

- São os atributos que vimos até agora

Exemplos

- **Classe Aluno**
 - String **matricula**: Cada aluno tem um número de matrícula único.
 - String **nome**: O nome é específico de cada aluno.
 - double[] **notas**: As notas de um aluno não afetam as notas de outro.
- **Classe Produto (em um e-commerce)**
 - String **sku**: O código identificador de um produto é único.
 - double **preco**: O preço pode variar para produtos diferentes, mesmo que sejam da mesma categoria.
 - int **quantidadeEmEstoque**: A quantidade disponível é uma característica do produto específico, não de todos os produtos da loja.
- **Classe ContaBancaria**
 - String **numeroDaConta**: Cada conta tem um número exclusivo.
 - double **saldo**: O saldo é a característica mais individual de uma conta.
 - Cliente **titular**: O dono de uma conta é específico para aquela instância.
- **Classe Postagem (em uma rede social como o X ou Instagram)**
 - String **textoConteudo**: O texto de uma postagem é único.
 - Usuario **autor**: Cada postagem é feita por um autor específico.
 - LocalDateTime **dataDePublicacao**: O momento exato da publicação pertence àquela postagem.
 - int **numeroDeCurtidas**: A contagem de "likes" é individual para cada postagem.
- **Classe ConexaoBancoDeDados**
 - String **sessionId**: O ID da sessão é único para cada conexão estabelecida.
 - long **tempoInicioConexao**: O momento em que a conexão foi aberta é específico dela.

Atributos de classe

São atributos de atribuição estática, e o valor atribuído está no contexto da classe

- Usamos 'static' na declaração
- É acessado através da classe e sua atribuição independe de parâmetros

Exemplos

- Classe Configuracao

- public static final String **URL_API** = "https://api.meusistema.com/v2/": Uma constante que define a URL base da API para toda a aplicação. Não faz sentido cada objeto ter uma cópia disso.
- public static final int **TIMEOUT_PADRAO** = 5000: O tempo máximo de espera padrão para requisições, compartilhado por todos os componentes que o utilizam.

- Classe Usuario

- private static int **contadorDeInstancias** = 0: Um contador que é incrementado no construtor para saber quantos objetos Usuario foram criados no total durante a execução do sistema.
- public static final int **TAMANHO_MINIMO_SENHA** = 8: Uma regra de negócio que se aplica a todos os usuários, sem exceção.

- Classe Calculadora

- public static final double **PI** = 3.14159265: Constantes matemáticas são o exemplo clássico. O valor de PI é universal e não pertence a uma "instância" de cálculo.

- Classe Logger (Padrão Singleton)

- private static Logger **instanciaUnica**: No padrão Singleton, a única instância da classe é armazenada em uma variável estática para garantir que todos os pontos do código usem o mesmo objeto de log.

- Classe Produto

- public static double **IMPOSTO_FEDERAL_PADRAO** = 0.0925: Se uma alíquota de imposto é a mesma para todos os produtos cadastrados, ela pode ser um atributo de classe. Se o imposto variar por produto, aí seria um atributo de instância.

Exemplo

```
1  package br.gov.sp.fatec.pg.oo;
2
3  public class Carro {
4
5      public static int numeroDeRodas = 4;
6
7      private String cor;
8      private String placa;
9
10     public void exibirInformacoes() {
11         // Acessando atributos de instância: Apenas pelo 'this'
12         System.out.println(x:"-----");
13         System.out.println("Carro com placa: " + this.placa);
14         System.out.println("Cor do carro (instância): " + this.cor);
15         // Acessando atributo de classe: via 'this' ou diretamente pela classe.
16         System.out.println("Número de rodas (classe): " + Carro.numeroDeRodas);
17         System.out.println(x:"-----\n");
18     }
19 }
```

Para que mais serviria isso?

```
3 public class Data {
4     private int dia;
5     private int mês;
6     private int ano;
7
8     private static String[] meses = {"JAN", "FEV", "MAR"
9                                     , "ABR", "MAI", "JUN"
10                                    , "JUL", "AGO", "SET"
11                                    , "OUT", "NOV", "DEZ"};
12
13     public String getNomeDoMês() {
14         return meses[this.mês-1];
15     }
```

Método

São funções declaradas dentro da classe e, por isso, encapsuladas em um contexto de objeto

- A declaração de um método possui cinco componentes:
 - **Modificador:** private, public ou outros que conheceremos mais tarde
 - **Tipo de retorno:** Tipo de dado retornado pelo método ou void caso seja procedural
 - **Nome do método:** Respeitando as mesmas regras de identificação de variáveis
 - **Lista de parâmetros:** Separados por ',' e entre parênteses, apresenta as variáveis do contexto de execução do método
 - **Corpo do método:** Delimitado pelo par de chaves '{' e '}', armazena o código executado pelo método

Exemplo

```
5 public class Pessoa {
6     String nome;
7     String cpf;
8     double altura;
9     double peso;
10    Pessoa mãe;
11    Pessoa pai;
12    LocalDate nascimento;
13
14    int getIdade() {
15        LocalDate dataAtual = LocalDate.now();
16        Period periodo = Period.between(nascimento, dataAtual);
17        return periodo.getYears();
18    }
19    public static void main(String[] args) {
```

Tarefa 01

- No projeto da aula anterior
 - Crie na classe Data um método chamado “getData” sem parâmetros que retorne uma String com a data no formato “DD/MM/AAAA”

Solução

```
public String getData() {  
    String out = "";  
    if (getDia() < 10) out += "0";  
    out += getDia() + "/";  
    if (getMês() < 10) out += "0";  
    out += getMês() + "/";  
    out += getAno();  
    return out;  
}
```

Tarefa 02

- No mesmo projeto:
 - Crie na classe Horário um método chamado “getHorário” sem parâmetros que retorne uma String com o horário no formato “HH:MM:SS”

Solução

```
public String getHorário() {  
    String out = "";  
    if(horas < 10) out += "0";  
    out += horas+":";  
    if(minutos < 10) out += "0";  
    out += minutos+":";  
    if(segundos < 10) out += "0";  
    out += segundos;  
    return out;  
}
```

Métodos construtores

Métodos especiais que são executados EXCLUSIVAMENTE quando um objeto é instanciado

- São declarados com o mesmo nome da classe, sem tipo de dados de retorno
- Podem ser sobrecarregados
- A execução ocorre quando executado um comando “new”

Exemplo

```
3  public class Data {  
4      private int dia;  
5      private int mês;  
6      private int ano;  
7  
8      public Data(){  
9          dia = 1;  
10         mês = 1;  
11         ano = 1970;  
12     }  
13  
14     public Data(int dia, int mês, int ano){  
15         this.dia = dia;  
16         this.mês = mês;  
17         this.ano = ano;  
18     }  
19 }
```

Tarefa 03

- No mesmo projeto:
 - Crie na classe Horário:
 - Um construtor sem parâmetros que inicialize o objeto com 00:00:00
 - Um construtor parametrizado que inicialize o objeto com os valores de horas, min e seg informados nos parâmetros
 - Use os métodos set ao invés de atribuir direto

Solução

```
3 public class Horario {
4     private int horas;
5     private int minutos;
6     private int segundos;
7
8     public Horario() {
9         horas=minutos=segundos = 0;
10    }
11    public Horario(int h, int m, int s) {
12        setHoras(h);
13        setMinutos(m);
14        setSegundos(s);
15    }
```

Tarefa 04

- No mesmo projeto:
 - Crie na classe Data:
 - Crie um método chamado isBissexto() que retorne true se o ano da data é bissexto ou false caso contrário
 - Crie na classe Horário:
 - Crie um método chamado isManhã() que retorne true se o horário está entre 6h e 12h e false caso contrário
 - Crie um método chamado isTarde() que retorne true se o horário está entre 12h e 18h e false caso contrário
 - Crie um método chamado isNoite() que retorne true se o horário está entre 18h e 0h e false caso contrário
 - Crie um método chamado isMadrugada() que retorne true se o horário está entre 0h e 6h e false caso contrário

Solução 1

```
69  [- public boolean isBisexto() {
70      if(ano % 400 == 0)
71          return true;
72      I else{
73          if(ano % 100 == 0)
74              return false;
75      76      else if(ano % 4 == 0)
77          return true;
78      79      else
79          return false;
80      }
80  }
```

Solução 2

```
57 public boolean isMadrugada() {  
58     if(horas >= 0 && horas < 6) return true;  
59     else return false;  
60 }  
61 public boolean isManhã() {  
62     if(horas >= 6 && horas < 12) return true;  
63     else return false;  
64 }  
65 public boolean isTarde() {  
66     if(horas >= 12 && horas < 18) return true;  
67     else return false;  
68 }  
69 public boolean isNoite() {  
70     if(horas >= 18 && horas < 24) return true;  
71     else return false;  
72 }
```

PARA FIXAR: QUIZ

Objetivo: Criar um projeto de aplicação que apresente um QUIZ (tema livre) com 10 questões:

- Os dados devem ser gravados em objetos de memória como ArrayList ou HashMap
- Ao final do QUIZ, deve ser apresentada a média de acertos